

Отчёт по лабораторной работе №6

**Установка и настройка системы управления базами данных
MariaDB**

Камбунду Паулине Де Лоурдеш Бранку

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
3	Вывод	30
4	Контрольные вопросы	33

Список иллюстраций

2.1	Установка MariaDB и просмотр конфигурационных файлов	9
2.2	Запуск службы MariaDB и проверка прослушивания порта 3306 .	10
2.3	Настройка параметров безопасности MariaDB с помощью mysql_secure_installation	12
2.4	Просмотр списка команд интерактивной оболочки MariaDB . . .	13
2.5	Просмотр системных баз данных MariaDB	14
2.6	Статус MariaDB до изменения кодировки сервера	18
2.7	Настройка кодировки UTF-8 в файле utf8.cnf	19
2.8	Статус MariaDB после установки UTF-8 в качестве кодировки по умолчанию	20
2.9	Создание базы addressbook, таблицы city и добавление записей .	23
2.10	Создание пользователя MariaDB, назначение прав и просмотр структуры таблицы city	25
2.11	Проверка базы addressbook и таблицы city с помощью mysqlshow	26
2.12	Создание и восстановление резервных копий базы данных addressbook	28
2.13	Скрипт автоматической установки и настройки MariaDB mysql.sh	29

Список таблиц

1 Цель работы

Приобретение практических навыков по установке и конфигурированию системы управления базами данных на примере программного обеспечения MariaDB.

2 Выполнение лабораторной работы

Для выполнения лабораторной работы на виртуальной машине `server` устанавливаю и настраиваю систему управления базами данных MariaDB. В процессе работы необходимо установить сервер и клиент MariaDB, выполнить первоначальную настройку безопасности, настроить кодировку UTF-8, создать тестовую базу данных `addressbook`, добавить в неё таблицу `city`, создать отдельного пользователя базы данных, выполнить резервное копирование и восстановление данных, а затем зафиксировать настройку MariaDB в скрипте автоматического развёртывания Vagrant.

На виртуальной машине `server` перехожу в режим суперпользователя и устанавливаю необходимые пакеты MariaDB:

```
dnf -y install mariadb mariadb-server
```

Параметр `-y` автоматически подтверждает установку всех необходимых пакетов. Пакет `mariadb` содержит клиентские средства для взаимодействия с сервером базы данных, а `mariadb-server` устанавливает серверную часть СУБД. Вместе с ними устанавливаются дополнительные зависимости, в частности клиентские утилиты, общие компоненты MariaDB, средства резервного копирования и серверные модули.

После завершения установки система выводит сообщение `Complete!`, подтверждающее успешную установку пакетов. В выводе также присутствуют установленные компоненты `mariadb`, `mariadb-client-utils`, `mariadb-server`, `mariadb-server-utils`, `mariadb-backup`, `mariadb-common` и другие необходимые зависимости.

Далее просматриваю содержимое каталога конфигурационных файлов MariaDB:

```
ls /etc/my.cnf.d/
```

В каталоге присутствуют файлы:

```
auth_gssapi.cnf
client.cnf
enable_encryption.preset
mariadb-server.cnf
mysql-clients.cnf
provider_bzip2.cnf
provider_lz4.cnf
provider_lzo.cnf
provider_snappy.cnf
spider.cnf
```

Файл `client.cnf` предназначен для параметров клиентских программ MariaDB. Файл `mariadb-server.cnf` содержит параметры серверного процесса MariaDB. Файл `mysql-clients.cnf` используется для конфигурации различных клиентских утилит MySQL/MariaDB. Файл `auth_gssapi.cnf` относится к модулю аутентификации GSSAPI. Файлы `provider_bzip2.cnf`, `provider_lz4.cnf`, `provider_lzo.cnf` и `provider_snappy.cnf` относятся к провайдерам алгоритмов сжатия. Файл `spider.cnf` содержит параметры движка Spider, а `enable_encryption.preset` используется для параметров, связанных с включением шифрования.

Основной конфигурационный файл просматриваю командой:

```
cat /etc/my.cnf
```

В нём содержится следующая конфигурация:

```
#  
# This group is read both both by the client and the server  
# use it for options that affect everything  
#  
[client-server]  
  
#  
# include all files from the config directory  
#  
!includedir /etc/my.cnf.d
```

Первая строка `#` является комментарием и сама по себе не задаёт параметров MariaDB. Строка `# This group is read both both by the client and the server` поясняет, что следующая группа параметров может использоваться как сервером, так и клиентскими программами. Строка `# use it for options that affect everything` уточняет назначение общей группы настроек.

Секция `[client-server]` предназначена для параметров, которые должны использоваться одновременно клиентской и серверной частями MariaDB.

Следующие строки, начинающиеся с символа `#`, также являются комментариями. Строка `# include all files from the config directory` поясняет дальнейшую директиву подключения дополнительных конфигурационных файлов.

Директива `!includedir /etc/my.cnf.d` заставляет MariaDB загружать дополнительные конфигурационные файлы из каталога `/etc/my.cnf.d`. Благодаря этому основную конфигурацию можно разделять на несколько файлов по назначению, не изменяя непосредственно `/etc/my.cnf` (см. рис. Рисунок 2.1).

```

Installed:
mariadb-3:10.11.18-1.el10_2.x86_64      mariadb-backup-3:10.11.18-1.el10_2.x86_64
mariadb-client-utils-3:10.11.18-1.el10_2.x86_64  mariadb-common-3:10.11.18-1.el10_2.noarch
mariadb-errmsg-3:10.11.18-1.el10_2.noarch      mariadb-gssapi-server-3:10.11.18-1.el10_2.x86_64
mariadb-server-3:10.11.18-1.el10_2.x86_64      mariadb-server-utils-3:10.11.18-1.el10_2.x86_64
mysql-selinux-1.0.14-1.el10.0.noarch           perl-DBD-MariaDB-1.23-10.el10.x86_64
perl-Sys-Hostname-1.25-515.el10_2.x86_64

Complete!
[root@server.paulinelourdes.net ~]# ls /etc/my.cnf.d/
auth_gssapi.cnf  enable_encryption.preset  mysql-clients.cnf  provider_lz4.cnf  provider_snappy.cnf
client.cnf      mariadb-server.cnf        provider_bzip2.cnf  provider_lzo.cnf  spider.cnf
[root@server.paulinelourdes.net ~]# cat /etc/my.cnf
#
# This group is read both both by the client and the server
# use it for options that affect everything
#
[client-server]
#
# include all files from the config directory
#
!includedir /etc/my.cnf.d
[root@server.paulinelourdes.net ~]# █

```

Рисунок 2.1: Установка MariaDB и просмотр конфигурационных файлов

После установки запускаю службу MariaDB:

```
systemctl start mariadb
```

Команда `systemctl start` запускает сервер базы данных в текущем сеансе работы операционной системы.

Чтобы MariaDB запускалась автоматически при последующих загрузках виртуальной машины, выполняю:

```
systemctl enable mariadb
```

При выполнении этой команды создаются необходимые символические ссылки для службы `mariadb.service`. В частности, служба связывается с целью `multi-user.target`, в результате чего сервер MariaDB автоматически запускается при переходе системы в многопользовательский режим.

После запуска MariaDB проверяю, прослушивает ли сервер стандартный порт MySQL/MariaDB. Сначала использую команду:

```
ss -tulpen | grep mysql
```

Поскольку название процесса в используемой версии системы отображается как `mariabdd`, дополнительно выполняю:

```
ss -tulpen | grep maria
```

Команда `ss` выводит информацию о сетевых сокетах. Параметр `-t` ограничивает вывод TCP-соединениями, `-u` добавляет UDP, `-l` показывает прослушива-

емые сокеты, -р выводит использующий сокет процесс, -е показывает дополнительную информацию, а -n запрещает преобразование числовых адресов и портов в символьные имена.

В выводе видно:

```
0.0.0.0:3306
[::]:3306
```

Следовательно, процесс `mariadb` успешно запущен и прослушивает стандартный порт MariaDB 3306 как для IPv4, так и для IPv6. Значение `LISTEN` показывает, что сервер ожидает входящие TCP-соединения (см. рис. Рисунок 2.2).

```
[root@server.paulinelourdes.net ~]# systemctl start mariadb
[root@server.paulinelourdes.net ~]# systemctl enable mariadb
Created symlink '/etc/systemd/system/mysql.service' -> '/usr/lib/systemd/system/mariadb.service'.
Created symlink '/etc/systemd/system/mysqld.service' -> '/usr/lib/systemd/system/mariadb.service'.
Created symlink '/etc/systemd/system/multi-user.target.wants/mariadb.service' -> '/usr/lib/systemd/system/mariadb.service'.
[root@server.paulinelourdes.net ~]# ss -tulpen | grep mysql
[root@server.paulinelourdes.net ~]# ss -tulpen | grep maria
tcp    LISTEN 0      80          0.0.0.0:3306      0.0.0.0:*        users:((("mariadb",pid=12444,fd=20)
                                     uid:27 ino:59044 sk:1a cgr
                                     ))
oup:/system.slice/mariadb.service <->
tcp    LISTEN 0      80          [::]:3306        [::]:*          users:((("mariadb",pid=12444,fd=21)
                                     uid:27 ino:59045 sk:28 cgr
                                     ))
oup:/system.slice/mariadb.service v6only:1 <->
[root@server.paulinelourdes.net ~]#
```

Рисунок 2.2: Запуск службы MariaDB и проверка прослушивания порта 3306

После запуска сервера выполняю первоначальную настройку безопасности MariaDB:

```
mysql_secure_installation
```

Данный скрипт позволяет изменить параметры, которые нежелательно оставлять в стандартной установке. Во время работы скрипта задаю пароль пользователя `root` MariaDB. Пользователь `root` в данном случае является администратором непосредственно СУБД и не является пользователем `root` операционной системы.

На запрос изменения пароля администратора выбираю положительный ответ и дважды ввожу новый пароль. Сообщения:

```
Password updated successfully!  
Reloading privilege tables..  
... Success!
```

подтверждают успешную смену пароля и обновление таблиц привилегий.
На запрос:

```
Remove anonymous users? [Y/n]
```

отвечаю Y. Это удаляет анонимные учётные записи MariaDB, которые в стандартной тестовой конфигурации позволяют подключаться к серверу без заранее созданного пользователя.

На запрос:

```
Disallow root login remotely? [Y/n]
```

также отвечаю Y. В результате запрещается удалённая аутентификация непосредственно под административной учётной записью root. Административный доступ к MariaDB сохраняется локально на сервере.

На запрос удаления тестовой базы данных также отвечаю Y. Тестовая база данных предназначена для первоначальных проверок и для рабочей конфигурации не требуется. В завершение настройки подтверждаю обновление таблиц привилегий. В результате удаляются небезопасные параметры стандартной установки MariaDB (см. рис. Рисунок 2.3).

```

You already have your root account protected, so you can safely answer 'n'.

Change the root password? [Y/n]
New password:
Re-enter new password:
Password updated successfully!
Reloading privilege tables..
... Success!

By default, a MariaDB installation has an anonymous user, allowing anyone
to log into MariaDB without having to have a user account created for
them. This is intended only for testing, and to make the installation
go a bit smoother. You should remove them before moving into a
production environment.

Remove anonymous users? [Y/n]
... Success!

Normally, root should only be allowed to connect from 'localhost'. This
ensures that someone cannot guess at the root password from the network.

Disallow root login remotely? [Y/n]
... Success!

By default, MariaDB comes with a database named 'test' that anyone can
access. This is also intended only for testing, and should be removed
before moving into a production environment.

Remove test database and access to it? [Y/n]

```

Рисунок 2.3: Настройка параметров безопасности MariaDB с помощью `mysql_secure_installation`

После настройки безопасности подключаюсь к MariaDB с правами администратора:

```
mysql -u root -p
```

Параметр `-u root` указывает имя пользователя базы данных, а параметр `-p` заставляет клиент запросить пароль перед подключением.

Для просмотра встроенной справки MariaDB из интерактивной оболочки выполняю:

```
\h
```

MariaDB выводит перечень доступных клиентских команд. В частности, команда `help` или `\h` открывает справку, `clear` или `\c` очищает текущий незавершённый запрос, `connect` или `\r` выполняет повторное подключение, `edit` или `\e` позволяет открыть текущую команду в редакторе, `exit` или `\q` завершает клиент MariaDB, а `status` или `\s` отображает состояние текущего соединения.

Команда `use` используется для переключения на другую базу данных, `source`

позволяет выполнить SQL-команды из файла, а system запускает команду оболочки операционной системы непосредственно из клиента MariaDB. Вывод справки подтверждает успешное подключение к серверу и возможность работы через интерактивную оболочку (см. рис. Рисунок 2.4).

```
General information about MariaDB can be found at
http://mariadb.org

List of all client commands:
Note that all text commands must be first on line and end with ';'
?          (\?) Synonym for 'help'.
charset    (\C) Switch to another charset. Might be needed for processing binlog with multi-byte charsets.
clear      (\c) Clear the current input statement.
connect    (\r) Reconnect to the server. Optional arguments are db and host.
delimiter  (\d) Set statement delimiter.
edit       (\e) Edit command with $EDITOR.
ego        (\G) Send command to MariaDB server, display result vertically.
exit       (\q) Exit mysql. Same as quit.
go         (\g) Send command to MariaDB server.
help       (\h) Display this help.
nopager    (\n) Disable pager, print to stdout.
notee      (\t) Don't write into outfile.
nowarning  (\W) Don't show warnings after every statement.
pager      (\P) Set PAGER [to_pager]. Print the query results via PAGER.
print      (\p) Print current command.
prompt     (\R) Change your mysql prompt.
quit       (\q) Quit mysql.
rehash     (\#) Rebuild completion hash.
sandbox    (\-) Disallow commands that access the file system (except \P without an argument and \e).
source     (\.) Execute an SQL script file. Takes a file name as an argument.
status     (\s) Get status information from the server.
system     (\!) Execute a system shell command.
tee        (\T) Set outfile [to_outfile]. Append everything into given outfile.
use        (\u) Use another database. Takes database name as argument.
warnings   (\W) Show warnings after every statement.

For server side help, type 'help contents'

MariaDB [(none)]>
```

Рисунок 2.4: Просмотр списка команд интерактивной оболочки MariaDB

Для просмотра имеющихся в системе баз данных выполняю SQL-запрос:

```
SHOW DATABASES;
```

MariaDB выводит четыре системные базы:

```
information_schema
mysql
performance_schema
sys
```

База `information_schema` предоставляет информацию о структуре баз данных, таблиц, столбцов, ограничений и других объектов СУБД. База `mysql` содержит системные таблицы MariaDB, включая сведения о пользователях, правах доступа и других внутренних настройках. База `performance_schema`

предназначена для сбора диагностической информации и показателей работы сервера. База sys предоставляет представления, упрощающие анализ информации, собранной системными таблицами и performance_schema.

После просмотра списка баз данных завершаю интерактивный сеанс командой:

```
exit;
```

Сообщение Bye подтверждает корректный выход из клиента MariaDB (см. рис. Рисунок 2.5).

```
MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database          |
+-----+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+-----+
4 rows in set (0.001 sec)

MariaDB [(none)]> exit;
Bye
[root@server.paulinelourdes.net ~]#
```

Рисунок 2.5: Просмотр системных баз данных MariaDB

Далее выполняю настройку кодировки символов. Снова подключаюсь к MariaDB с правами администратора и выполняю:

```
status
```

Перед изменением конфигурации сервер выводит сведения о текущем соединении и параметрах MariaDB.

Строка:

```
mysql Ver 15.1 Distrib 10.11.18-MariaDB, for Linux (x86_64) using EditLine wrapper
```

указывает версию клиентской программы MariaDB — 10.11.18, архитектуру x86_64 и используемый механизм редактирования командной строки.

Строка:

```
Connection id: 12
```

содержит уникальный идентификатор текущего соединения с сервером.

Поле:

```
Current database:
```

не содержит имени базы данных, поскольку после подключения ни одна пользовательская база ещё не была выбрана командой USE.

Строка:

```
Current user: root@localhost
```

показывает, что соединение выполняется от имени пользователя root с локальной машины.

Строка:

```
SSL: Not in use
```

показывает, что текущее локальное подключение к MariaDB не использует SSL/TLS.

Строка:

```
Current pager: stdout
```

означает, что результаты запросов выводятся непосредственно в стандартный поток вывода терминала.

Строка:

```
Using outfile: ''
```

показывает, что вывод не перенаправляется в отдельный файл.

Строка:

```
Using delimiter: ;
```

указывает, что символ ; используется как признак окончания SQL-команды.

Строка:

```
Server: MariaDB
```

определяет тип используемого сервера базы данных.

Строка:

```
Server version: 10.11.18-MariaDB MariaDB Server
```

содержит точную версию серверной части MariaDB.

Строка:

```
Protocol version: 10
```

показывает версию протокола, используемого клиентом и сервером MariaDB.

Строка:

```
Connection: Localhost via UNIX socket
```

означает, что локальное подключение выполнено не через TCP-порт 3306, а через UNIX-сокеты.

До изменения конфигурации строки:

```
Server charsetset: latin1
Db      charsetset: latin1
Client charsetset: utf8mb3
Conn.  charsetset: utf8mb3
```

показывают, что кодировкой сервера и базы данных по умолчанию является `latin1`, тогда как клиент и текущее соединение используют `utf8mb3`.

Строка:

```
UNIX socket: /var/lib/mysql/mysql.sock
```

задаёт путь к UNIX-сокету локального сервера MariaDB.

Поле:

```
Uptime: 3 min 48 sec
```

показывает время, прошедшее с момента последнего запуска службы MariaDB.

В нижней части вывода находятся текущие статистические показатели сервера:

```
Threads: 1
Questions: 23
Slow queries: 0
Opens: 20
Open tables: 13
Queries per second avg: 0.100
```

`Threads` показывает количество активных потоков соединений, `Questions` — количество обработанных сервером команд, `Slow queries` — число медленных

запросов, Opens — количество операций открытия таблиц, Open tables — количество открытых в данный момент таблиц, а Queries per second avg — среднее число запросов в секунду (см. рис. Рисунок 2.6).

```
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 12
Server version: 10.11.18-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> status
-----
mysql Ver 15.1 Distrib 10.11.18-MariaDB, for Linux (x86_64) using Editline wrapper

Connection id:          12
Current database:
Current user:           root@localhost
SSL:                   Not in use
Current pager:          stdout
Using outfile:          ''
Using delimiter:        ;
Server:                 MariaDB
Server version:         10.11.18-MariaDB MariaDB Server
Protocol version:       10
Connection:             Localhost via UNIX socket
Server characterset:    latin1
Db characterset:        latin1
Client characterset:    utf8mb3
Conn. characterset:     utf8mb3
UNIX socket:            /var/lib/mysql/mysql.sock
Uptime:                 3 min 48 sec

Threads: 1  Questions: 23  Slow queries: 0  Opens: 20  Open tables: 13  Queries per second avg: 0.100
-----
MariaDB [(none)]> █
```

Рисунок 2.6: Статус MariaDB до изменения кодировки сервера

Для установки UTF-8 в качестве кодировки по умолчанию перехожу в каталог конфигурации MariaDB:

```
cd /etc/my.cnf.d
```

Создаю новый конфигурационный файл:

```
touch utf8.cnf
```

Открываю файл utf8.cnf на редактирование и добавляю в него:

```
[client]
default-character-set = utf8

[mysqld]
character-set-server = utf8
```

Секция [client] содержит параметры клиентских программ MariaDB. Директива:

```
default-character-set = utf8
```

задаёт кодировку UTF-8 по умолчанию для клиента.

Секция [mysqld] относится к серверному процессу MariaDB. Директива:

```
character-set-server = utf8
```

назначает UTF-8 основной кодировкой сервера и создаваемых объектов базы данных, если иная кодировка не была явно задана при создании (см. рис. Рисунок 2.7).

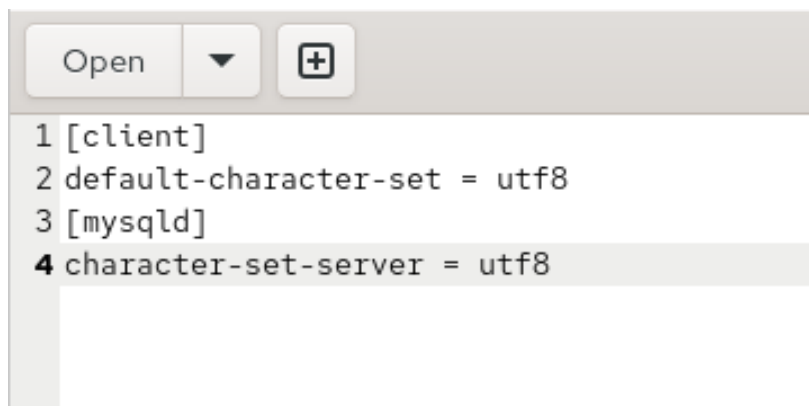


Рисунок 2.7: Настройка кодировки UTF-8 в файле utf8.cnf

Для применения новой конфигурации перезапускаю службу:

```
systemctl restart mariadb
```

После перезапуска снова подключаюсь к серверу:

```
mysql -u root -p
```

и повторно выполняю:

```
status
```

В новом выводе MariaDB строки кодировок имеют вид:

```
Server characterset: utf8mb3
Db      characterset: utf8mb3
```

```
Client charset: utf8mb3
```

```
Conn. charset: utf8mb3
```

По сравнению с предыдущим состоянием изменились параметры `Server charset` и `Db charset`: вместо `latin1` теперь используется `utf8mb3`. В MariaDB параметр `utf8` в данной конфигурации отображается как `utf8mb3`, то есть трёхбайтовая реализация UTF-8. Клиент и соединение также используют `utf8mb3`.

Изменился и идентификатор соединения — после перезапуска отображается `Connection id: 3`. Значение `Uptime` составляет 16 сек, что подтверждает недавний перезапуск службы. Счётчики запросов, открытий таблиц и другие статистические показатели также начинаются заново после перезапуска сервера.

Следовательно, созданный файл `/etc/my.cnf.d/utf8.cnf` успешно прочитан MariaDB, и UTF-8 установлена в качестве кодировки сервера и базы данных по умолчанию (см. рис. Рисунок 2.8).

```
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 3
Server version: 10.11.18-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> status
-----
mysql Ver 15.1 Distrib 10.11.18-MariaDB, for Linux (x86_64) using Editline wrapper

Connection id:          3
Current database:
Current user:            root@localhost
SSL:                    Not in use
Current pager:           stdout
Using outfile:           ''
Using delimiter:        ;
Server:                 MariaDB
Server version:          10.11.18-MariaDB MariaDB Server
Protocol version:        10
Connection:             Localhost via UNIX socket
Server charset:          utf8mb3
Db charset:              utf8mb3
Client charset:          utf8mb3
Conn. charset:           utf8mb3
UNIX socket:             /var/lib/mysql/mysql.sock
Uptime:                 16 sec

Threads: 1  Questions: 4  Slow queries: 0  Opens: 17  Open tables: 10  Queries per second avg: 0.250
-----
MariaDB [(none)]>
```

Рисунок 2.8: Статус MariaDB после установки UTF-8 в качестве кодировки по умолчанию

Следующим этапом создаю тестовую базу данных addressbook. Подключившись к MariaDB с правами администратора, выполняю:

```
CREATE DATABASE addressbook CHARACTER SET utf8 COLLATE utf8_general_ci;
```

Команда `CREATE DATABASE addressbook` создаёт новую базу данных с именем addressbook. Конструкция `CHARACTER SET utf8` явно задаёт кодировку UTF-8, а `COLLATE utf8_general_ci` определяет правила сравнения и сортировки строк. Суффикс ci означает case insensitive, то есть сравнение символов выполняется без учёта регистра.

Сообщение:

```
Query OK, 1 row affected
```

подтверждает успешное создание базы данных.

Переключаюсь на созданную базу:

```
USE addressbook;
```

После выполнения команды MariaDB сообщает:

```
Database changed
```

Проверяю наличие таблиц:

```
SHOW TABLES;
```

На данном этапе выводится:

```
Empty set
```

поскольку база только что создана и пользовательские таблицы в ней отсутствуют.

Создаю таблицу city:

```
CREATE TABLE city(name VARCHAR(40), city VARCHAR(40));
```

Поле name имеет тип VARCHAR(40) и предназначено для хранения фамилии сотрудника длиной до 40 символов. Поле city также имеет тип VARCHAR(40) и используется для хранения названия города.

После создания таблицы добавляю три записи:

```
INSERT INTO city(name, city) VALUES ('Иванов', 'Москва');  
INSERT INTO city(name, city) VALUES ('Петров', 'Сочи');  
INSERT INTO city(name, city) VALUES ('Сидоров', 'Дубна');
```

Для каждой команды MariaDB выводит сообщение Query OK, 1 row affected, означающее успешное добавление одной строки.

Для проверки содержимого таблицы выполняю:

```
SELECT * FROM city;
```

Символ * указывает на необходимость вывести все столбцы таблицы. Результат запроса имеет вид:

```
+-----+-----+  
| name  | city  |  
+-----+-----+  
| Иванов | Москва |  
| Петров | Сочи   |  
| Сидоров | Дубна  |  
+-----+-----+  
3 rows in set
```

В результате отображаются все три добавленные записи: Иванов работает в Москве, Петров — в Сочи, а Сидоров — в Дубне. Строка 3 rows in set подтверждает наличие трёх записей в таблице (см. рис. Рисунок 2.9).

```

MariaDB [(none)]>
MariaDB [(none)]> CREATE DATABASE addressbook CHARACTER SET utf8 COLLATE utf8_general_ci;
Query OK, 1 row affected (0.000 sec)

MariaDB [(none)]> USE addressbook;
Database changed
MariaDB [addressbook]> SHOW TABLES;
Empty set (0.000 sec)

MariaDB [addressbook]> CREATE TABLE city (name VARCHAR(40), city VARCHAR(40));
Query OK, 0 rows affected (0.014 sec)

MariaDB [addressbook]> INSERT INTO city(name, city) VALUES ('Иванов','Москва');
Query OK, 1 row affected (0.001 sec)

MariaDB [addressbook]> INSERT INTO city(name, city) VALUES ('Петров','Сочи');
Query OK, 1 row affected (0.001 sec)

MariaDB [addressbook]> INSERT INTO city(name, city) VALUES ('Сидоров','Дубна');
Query OK, 1 row affected (0.003 sec)

MariaDB [addressbook]> SELECT * FROM city;
+-----+-----+
| name      | city    |
+-----+-----+
| Иванов    | Москва  |
| Петров    | Сочи    |
| Сидоров   | Дубна   |
+-----+-----+
3 rows in set (0.000 sec)

MariaDB [addressbook]> █

```

Рисунок 2.9: Создание базы addressbook, таблицы city и добавление записей

Для работы с базой данных от отдельной учётной записи создаю пользователя MariaDB с именем, соответствующим используемому логину:

```
CREATE USER paulinelourdes@'%' IDENTIFIED BY '123456';
```

Имя paulinelourdes задаёт пользователя базы данных. Символ % в части узла означает шаблон любого сетевого узла, с которого может сопоставляться учётная запись при соблюдении остальных настроек доступа. Параметр IDENTIFIED BY '123456' задаёт пароль пользователя.

MariaDB сообщает:

```
Query OK, 0 rows affected
```

что подтверждает успешное создание учётной записи.

Предоставляю пользователю необходимые права на базу addressbook:

```
GRANT SELECT,INSERT,UPDATE,DELETE ON addressbook.* TO paulinelourdes@'%';
```

Привилегия `SELECT` разрешает чтение данных, `INSERT` — добавление новых строк, `UPDATE` — изменение существующих данных, а `DELETE` — удаление записей. Конструкция `addressbook.*` означает, что перечисленные разрешения распространяются на все таблицы базы данных `addressbook`.

После назначения прав выполняю:

```
FLUSH PRIVILEGES;
```

Команда заставляет MariaDB обновить используемые сведения о правах доступа.

Затем просматриваю структуру таблицы:

```
DESCRIBE city;
```

Результат имеет вид:

Field	Type	Null	Key	Default	Extra
name	varchar(40)	YES		NULL	
city	varchar(40)	YES		NULL	

Поле `Field` содержит имена столбцов `name` и `city`. В столбце `Type` видно, что оба поля имеют тип `varchar(40)`. Значение `YES` в столбце `Null` показывает, что для обоих полей допускается значение `NULL`. Столбец `Key` пуст, поскольку первичные и другие индексы для таблицы не создавались. В поле `Default` указано `NULL`, то есть при отсутствии явно заданного значения используется `NULL`. Поле `Extra` также пусто, поскольку дополнительные свойства для столбцов не назначались.

После проверки выхожу из MariaDB командой:

```
quit
```

Сообщение Bye подтверждает завершение интерактивного сеанса (см. рис. Рисунок 2.10).

```
MariaDB [addressbook]>
MariaDB [addressbook]> CREATE USER paulinelourdes@'%' IDENTIFIED BY '123456';
Query OK, 0 rows affected (0.001 sec)

MariaDB [addressbook]> GRANT SELECT,INSERT,UPDATE,DELETE ON addressbook.* TO paulinelourdes@'%';
Query OK, 0 rows affected (0.002 sec)

MariaDB [addressbook]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.000 sec)

MariaDB [addressbook]> DESCRIBE city;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| name  | varchar(40)   | YES  |     | NULL    |       |
| city  | varchar(40)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.001 sec)

MariaDB [addressbook]> quit
Bye
[root@server.paulinelourdes.net my.cnf.d]#
```

Рисунок 2.10: Создание пользователя MariaDB, назначение прав и просмотр структуры таблицы city

Дополнительно проверяю созданные объекты с помощью утилиты mysqlshow.

Для просмотра списка баз данных выполняю:

```
mysqlshow -u root -p
```

Параметр -u root задаёт пользователя root, а -p запрашивает пароль. В результате отображаются:

```
addressbook
information_schema
mysql
performance_schema
sys
```

По сравнению с первоначальным состоянием появился новый объект addressbook, что подтверждает успешное создание пользовательской базы данных.

Для просмотра таблиц внутри addressbook выполняю:

```
mysqlshow -u root -p addressbook
```

В результате отображается таблица:

```
Database: addressbook
```

```
+-----+
| Tables |
+-----+
| city   |
+-----+
```

Затем выполняю аналогичную проверку от имени созданного пользователя:

```
mysqlshow -u paulinelourdes -p addressbook
```

После ввода пароля также отображается таблица city. Следовательно, созданный пользователь имеет достаточные права для обращения к базе addressbook и просмотра её объектов (см. рис. Рисунок 2.11).

```
[root@server.paulinelourdes.net my.cnf.d]# mysqlshow -u root -p
Enter password:
+-----+
| Databases |
+-----+
| addressbook |
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
[root@server.paulinelourdes.net my.cnf.d]# mysqlshow -u root -p addressbook
Enter password:
Database: addressbook
+-----+
| Tables |
+-----+
| city |
+-----+
[root@server.paulinelourdes.net my.cnf.d]# mysqlshow -u paulinelourdes -p addressbook
Enter password:
Database: addressbook
+-----+
| Tables |
+-----+
| city |
+-----+
[root@server.paulinelourdes.net my.cnf.d]#
```

Рисунок 2.11: Проверка базы addressbook и таблицы city с помощью mysqlshow

Для защиты данных от потери выполняю резервное копирование базы addressbook. Сначала создаю каталог для хранения резервных копий:

```
mkdir -p /var/backup
```

Параметр `-p` создаёт указанный каталог и при необходимости отсутствующие родительские каталоги, не выдавая ошибку, если каталог уже существует.

Создаю обычную SQL-копию базы данных:

```
mysqldump -u root -p addressbook > /var/backup/addressbook.sql
```

Утилита `mysqldump` формирует текстовый набор SQL-команд, позволяющих в дальнейшем воссоздать структуру базы и содержащиеся в ней данные. Параметр `-u root` задаёт административного пользователя, `-p` запрашивает его пароль, а `addressbook` определяет копируемую базу. Оператор `>` перенаправляет полученный SQL-дамп в файл `/var/backup/addressbook.sql`.

После этого формирую сжатую резервную копию:

```
mysqldump -u root -p addressbook | gzip > /var/backup/addressbook.sql.gz
```

В данном случае вывод `mysqldump` через конвейер `|` передаётся программе `gzip`, которая сжимает данные. Сжатый результат записывается в файл `/var/backup/addressbook.sql.gz`.

Для создания резервной копии, в имени которой содержится время формирования, выполняю:

```
mysqldump -u root -p addressbook | gzip > $(date +%Y%m%d.%H%M%S).sql.gz
```

Команда `date` формирует имя файла по шаблону. Последовательность `%Y` заменяется четырёхзначным годом, `%m` — номером месяца, `%d` — днём месяца, `%H` — часом, `%M` — минутами, а `%S` — секундами. Это позволяет хранить несколько резервных копий, не перезаписывая предыдущие.

Для проверки возможности восстановления использую несжатую копию:

```
mysql -u root -p addressbook < /var/backup/addressbook.sql
```

Оператор < передаёт содержимое файла `addressbook.sql` на стандартный ввод клиента `mysql`. Содержащиеся в дампе SQL-команды выполняются для базы `addressbook`, восстанавливая её структуру и данные.

Затем проверяю восстановление из сжатой резервной копии:

```
zcat /var/backup/addressbook.sql.gz | mysql -u root -p addressbook
```

Команда `zcat` распаковывает содержимое `.gz` непосредственно в стандартный поток вывода, после чего через конвейер оно передаётся клиенту `MariaDB`. Отсутствие сообщений об ошибках после обеих команд восстановления подтверждает корректное чтение созданных резервных копий (см. рис. Рисунок 2.12).

```
[root@server.paulinelourdes.net my.cnf.d]#
[root@server.paulinelourdes.net my.cnf.d]# mkdir -p /var/backup
[root@server.paulinelourdes.net my.cnf.d]# mysqldump -u root -p addressbook > /var/backup/addressbook.sql
Enter password:
[root@server.paulinelourdes.net my.cnf.d]# mysqldump -u root -p addressbook | gzip > /var/backup/addressbook.sql.gz
Enter password:
[root@server.paulinelourdes.net my.cnf.d]# mysqldump -u root -p addressbook | gzip > $(date +%Y%m%d.%H%M%S).sql.gz
Enter password:
[root@server.paulinelourdes.net my.cnf.d]# mysql -u root -p addressbook < /var/backup/addressbook.sql
Enter password:
[root@server.paulinelourdes.net my.cnf.d]# zcat /var/backup/addressbook.sql.gz | mysql -u root -p addressbook
Enter password:
[root@server.paulinelourdes.net my.cnf.d]#
```

Рисунок 2.12: Создание и восстановление резервных копий базы данных `addressbook`

После ручной настройки `MariaDB` фиксирую выполненные действия в системе автоматического развёртывания `Vagrant`.

На виртуальной машине `server` перехожу в каталог:

```
cd /vagrant/provision/server
```

Создаю структуру каталогов для хранения конфигурации `MariaDB` и резервных копий.

Копирую созданный конфигурационный файл кодировки UTF-8. Также копирую резервные копии базы данных.

После этого создаю файл автоматического развёртывания:

```
1  #!/bin/bash
2  echo "Provisioning script $0"
3  systemctl restart named
4  echo "Install needed packages"
5  dnf -y install mariadb mariadb-server
6  echo "Copy configuration files"
7  cp -R /vagrant/provision/server/mysql/etc/* /etc
8  mkdir -p /var/backup
9  cp -R /vagrant/provision/server/mysql/var/backup/* /var/backup
10 echo "Start mysql service"
11 systemctl enable mariadb
12 systemctl start mariadb
13 if [[ ! -d /var/lib/mysql/mysql ]]
14 then
15     echo "Securing mariadb"
16     mysql_secure_installation <<EOF
17     y
18     123456
19     123456
20     y
21     y
22     y
23     y
24     EOF
25     echo "Create database"
26     mysql -u root -p123456 <<EOF
27     CREATE DATABASE addressbook CHARACTER SET utf8 COLLATE utf8_general_ci;
28     EOF
29     mysql -u root -p123456 addressbook < /var/backup/addressbook.sql
30 fi
```

Рисунок 2.13: Скрипт автоматической установки и настройки MariaDB mysql.sh

3 Вывод

В ходе лабораторной работы была выполнена установка, первоначальная настройка и автоматизация развёртывания системы управления базами данных MariaDB на виртуальной машине server. Основное внимание уделялось настройке безопасности сервера баз данных, установке кодировки UTF-8, созданию пользовательской базы данных, управлению правами доступа, резервному копированию и последующему восстановлению данных.

Для работы с MariaDB были установлены клиентская и серверная части СУБД с помощью команды:

```
dnf -y install mariadb mariadb-server
```

После установки служба mariadb была запущена и добавлена в автозагрузку посредством `systemctl start mariadb` и `systemctl enable mariadb`. Проверка сетевых сокетов показала, что серверный процесс `mariabdd` прослушивает стандартный TCP-порт 3306, используемый MariaDB и MySQL для сетевых подключений.

Первоначальная настройка безопасности была выполнена с использованием программы `mysql_secure_installation`. В процессе настройки был установлен пароль административного пользователя `root` MariaDB, удалены анонимные пользователи и тестовая база данных, запрещён удалённый вход под учётной записью `root`, а изменения таблиц привилегий были применены к работающему серверу. Это позволило исключить ряд небезопасных параметров, присутствующих в первоначальной конфигурации СУБД.

Были рассмотрены основные конфигурационные файлы MariaDB. Глав-

ный файл `/etc/my.cnf` подключает дополнительные файлы из каталога `/etc/my.cnf.d`. Для изменения кодировки был создан отдельный файл `/etc/my.cnf.d/utf8.cnf`, в котором для клиентской и серверной частей была задана кодировка UTF-8. После перезапуска MariaDB команда `status` показала изменение серверной кодировки с `latin1` на `utf8mb3`, используемую установленной версией MariaDB для значения `utf8`.

Для хранения тестовых данных была создана база:

`addressbook`

при помощи запроса:

```
CREATE DATABASE addressbook CHARACTER SET utf8 COLLATE utf8_general_ci;
```

В базе была создана таблица `city`, содержащая строковые поля `name` и `city`. В неё были добавлены сведения об Иванове, Петрове и Сидорове с соответствующими городами Москва, Сочи и Дубна. Запрос `SELECT * FROM city;` подтвердил наличие всех трёх записей.

Для разграничения доступа был создан отдельный пользователь MariaDB `raulinelourdes`, которому были предоставлены права `SELECT`, `INSERT`, `UPDATE` и `DELETE` на все таблицы базы `addressbook`. Это позволяет данному пользователю просматривать, добавлять, изменять и удалять данные, не предоставляя ему полных административных полномочий сервера MariaDB. Структура таблицы была проверена командой `DESCRIBE city`, а наличие базы данных и таблицы дополнительно подтверждено с помощью утилиты `mysqlshow`.

На следующем этапе была рассмотрена организация резервного копирования. С помощью `mysqldump` была создана обычная SQL-копия базы `/var/backup/addressbook.sql`, а также её сжатые варианты в формате `gzip`. Дополнительно была создана резервная копия, имя которой содержит дату и время формирования. Возможность восстановления была проверена посредством передачи содержимого SQL-файла клиенту `mysql`, а для сжатой копии использовалась связка `zcat` и `mysql`.

В завершение выполненные действия были перенесены в механизм авто-

матического развёртывания Vagrant. Конфигурационный файл `utf8.cnf` и резервные копии базы были сохранены в каталоге `/vagrant/provision/server/mysql`, а в сценарии `/vagrant/provision/server/mysql.sh` была зафиксирована установка MariaDB, копирование конфигурационных файлов, запуск и включение службы, первоначальная настройка безопасности, создание базы `addressbook` и восстановление её содержимого из резервной копии. Соответствующий provisioning-сценарий был подключён в `Vagrantfile`.

В результате лабораторной работы была подготовлена работоспособная конфигурация MariaDB, использующая UTF-8, содержащая тестовую базу `addressbook` и поддерживающая разграничение пользовательских прав, резервное копирование и восстановление данных. Автоматизация средствами Vagrant позволяет воспроизвести необходимые настройки при повторном создании виртуальной машины.

4 Контрольные вопросы

1. Какая команда отвечает за настройки безопасности в MariaDB?

Для выполнения первоначальной настройки безопасности MariaDB используется команда:

```
mysql_secure_installation
```

Она запускает интерактивный сценарий, позволяющий изменить основные параметры безопасности стандартной установки MariaDB.

С его помощью можно:

- установить или изменить пароль административного пользователя root MariaDB;
- удалить анонимные учётные записи;
- запретить удалённый вход пользователя root;
- удалить тестовую базу данных;
- обновить таблицы привилегий.

В лабораторной работе команда запускалась следующим образом:

```
mysql_secure_installation
```

После запуска программа последовательно задаёт вопросы и предлагает отвечать Y или n.

Следует различать пользователя root операционной системы и пользователя root MariaDB. Это две разные учётные записи. Первая используется для администрирования Linux, а вторая является административной учётной записью непосредственно сервера баз данных.

2. Как настроить MariaDB для доступа через сеть?

Для сетевого доступа необходимо обеспечить сразу несколько условий: MariaDB должна прослушивать сетевой интерфейс, сетевой порт должен быть разрешён межсетевым экраном, а подключающийся пользователь должен иметь право входа с соответствующего узла.

Стандартным TCP-портом MariaDB является:

3306

Параметры сетевого прослушивания задаются в секции [mysqld] конфигурационных файлов MariaDB, например в /etc/my.cnf.d/mariadb-server.cnf.

Для приёма подключений на всех сетевых интерфейсах может использоваться:

```
[mysqld]  
bind-address=0.0.0.0
```

Более безопасным вариантом является указание конкретного IP-адреса интерфейса сервера, например:

```
[mysqld]  
bind-address=192.168.56.10
```

После изменения конфигурации необходимо перезапустить MariaDB:

```
systemctl restart mariadb
```

Проверить прослушиваемый порт можно командой:

```
ss -tulpen | grep 3306
```

Кроме того, требуется создать пользователя, которому разрешены удалённые подключения. Например:

```
CREATE USER user@'%' IDENTIFIED BY 'password';
```

Символ % обозначает возможность сопоставления подключения с любого узла. После этого пользователю необходимо предоставить необходимые права:

```
GRANT SELECT,INSERT,UPDATE,DELETE ON addressbook.* TO user@'%';
```

и применить изменения:

```
FLUSH PRIVILEGES;
```

В рабочей системе вместо % предпочтительно ограничивать источник определённым узлом или сетью, если доступ со всех адресов не требуется.

Если на сервере используется `firewalld`, необходимо также разрешить входящие соединения к MariaDB, например:

```
firewall-cmd --add-service=mysql --permanent  
firewall-cmd --reload
```

Либо можно разрешить непосредственно TCP-порт:

```
firewall-cmd --add-port=3306/tcp --permanent  
firewall-cmd --reload
```

Следовательно, для работы MariaDB через сеть необходимо настроить сетевое прослушивание сервера, права пользователя и правила межсетевого экрана.

3. Какая команда позволяет получить обзор доступных баз данных после входа в среду оболочки MariaDB?

Для просмотра доступных баз данных после входа в интерактивную оболочку MariaDB используется SQL-команда:

```
SHOW DATABASES;
```

Например:

```
MariaDB [(none)]> SHOW DATABASES;
```

В первоначально настроенной системе могут отображаться системные базы:

```
information_schema  
mysql  
performance_schema  
sys
```

После выполнения лабораторной работы дополнительно появляется база:
addressbook

База `information_schema` содержит метаданные о структуре объектов СУБД, `mysql` — внутреннюю системную информацию и сведения о пользователях и

привилегиях, `performance_schema` — информацию для анализа работы сервера, а `sys` предоставляет удобные системные представления.

Аналогичный обзор вне интерактивной оболочки можно получить утилитой:

```
mysqlshow -u root -p
```

4. Какая команда позволяет узнать, какие таблицы доступны в базе данных?

Для просмотра таблиц текущей выбранной базы данных используется команда:

```
SHOW TABLES;
```

Сначала необходимо выбрать базу командой `USE`. Например:

```
USE addressbook;  
SHOW TABLES;
```

Если в базе имеется созданная в лабораторной работе таблица `city`, результат будет содержать:

```
+-----+  
| Tables_in_addressbook |  
+-----+  
| city                  |  
+-----+
```

Посмотреть список таблиц можно и без предварительной команды `USE`, указав имя базы:

```
SHOW TABLES FROM addressbook;
```

Из командной оболочки операционной системы аналогичную проверку можно выполнить с помощью:

```
mysqlshow -u root -p addressbook
```

5. Какая команда позволяет узнать, какие поля доступны в таблице?

Для просмотра структуры таблицы и содержащихся в ней полей применяется команда:

```
DESCRIBE имя_таблицы;
```

Для созданной в лабораторной работе таблицы используется:

```
DESCRIBE city;
```

Сокращённая форма команды:

```
DESC city;
```

Результат содержит сведения о каждом поле таблицы:

```
Field
Type
Null
Key
Default
Extra
```

Для таблицы city отображаются два поля:

```
name
city
```

Оба имеют тип:

`VARCHAR(40)`

Field показывает имя поля, Type — тип данных, Null определяет возможность хранения значения NULL, Key содержит информацию об индексах и ключах, Default показывает значение по умолчанию, а Extra — дополнительные свойства столбца.

Для получения аналогичной информации также существует команда:

```
SHOW COLUMNS FROM city;
```

6. Какая команда позволяет узнать, какие записи доступны в таблице?

Для просмотра записей таблицы используется оператор SQL `SELECT`.

Чтобы получить все строки и все столбцы таблицы `city`, используется:

```
SELECT * FROM city;
```

Символ `*` обозначает выбор всех доступных столбцов.

В созданной во время лабораторной работы таблице результат имеет следующий смысл:

```
+-----+-----+
| name  | city  |
+-----+-----+
| Иванов | Москва |
| Петров | Сочи  |
| Сидоров | Дубна |
+-----+-----+
```

При необходимости можно выбрать только определённые поля. Например:

```
SELECT name FROM city;
```

Можно также выбирать только записи, соответствующие условию:

```
SELECT * FROM city WHERE city='Москва';
```

В этом случае MariaDB вернёт только строки, в которых значение столбца city равно Москва.

7. Как удалить запись из таблицы?

Для удаления записей из таблицы используется SQL-команда DELETE.

Например, чтобы удалить из таблицы city запись о сотруднике Сидорове, можно выполнить:

```
DELETE FROM city WHERE name='Сидоров';
```

Условие:

```
WHERE name='Сидоров'
```

определяет, какие именно строки должны быть удалены.

После удаления результат можно проверить:

```
SELECT * FROM city;
```

При необходимости условие можно составить сразу по нескольким полям:

```
DELETE FROM city  
WHERE name='Сидоров' AND city='Дубна';
```

Особенно важно при использовании DELETE обращать внимание на наличие условия WHERE.

Команда:

```
DELETE FROM city;
```

не содержит условия и поэтому удалит **все записи** из таблицы city, хотя сама таблица и её структура сохранятся.

Для полного удаления самой таблицы применяется уже другая команда:

```
DROP TABLE city;
```

Следовательно, DELETE используется для удаления строк, а DROP TABLE — для удаления самой таблицы как объекта базы данных.

8. Где расположены файлы конфигурации MariaDB? Что можно настроить с их помощью?

В использованной в лабораторной работе системе основной конфигурационный файл MariaDB расположен по адресу:

```
/etc/my.cnf
```

В нём присутствует директива:

```
!includedir /etc/my.cnf.d
```

Она подключает дополнительные конфигурационные файлы из каталога:

```
/etc/my.cnf.d/
```

Благодаря этому параметры MariaDB могут быть разделены между несколькими файлами. Например, во время лабораторной работы был создан файл:

```
/etc/my.cnf.d/utf8.cnf
```

со следующим содержимым:

```
[client]
default-character-set = utf8

[mysqld]
character-set-server = utf8
```

Секция `[client]` определяет параметры клиентских приложений, а `[mysqld]` — параметры серверного процесса MariaDB.

С помощью конфигурационных файлов можно настраивать большое количество параметров сервера, в том числе:

- кодировку символов;
- правила сортировки и сравнения строк;
- сетевые интерфейсы, на которых работает сервер;
- TCP-порт MariaDB;
- расположение каталога баз данных;
- параметры журналирования;
- размеры различных буферов;
- число допустимых соединений;
- тайм-ауты соединений;
- параметры хранения и обработки данных;
- настройки различных механизмов хранения;
- SSL/TLS;
- некоторые параметры безопасности;
- параметры клиентских программ.

Например, кодировка сервера может быть задана:

```
[mysqld]
```

```
character-set-server = utf8
```

Порт можно определить параметром:

```
port = 3306
```

а адрес сетевого интерфейса:

```
bind-address = 0.0.0.0
```

После изменения серверных настроек обычно требуется перезапуск MariaDB:

```
systemctl restart mariadb
```

9. Где располагаются файлы с базами данных MariaDB?

В стандартной установке MariaDB в рассматриваемой Linux-системе каталог данных находится по адресу:

```
/var/lib/mysql
```

Внутри него находятся системные файлы MariaDB, файлы механизмов хранения и каталоги отдельных баз данных.

Например, для базы:

```
addressbook
```

обычно используется соответствующий каталог внутри каталога данных:

```
/var/lib/mysql/addressbook
```

Также в `/var/lib/mysql` находятся служебные объекты MariaDB. В лабораторной работе наличие каталога:

```
/var/lib/mysql/mysql
```

используется в скрипте `mysql.sh` для проверки того, была ли уже выполнена инициализация MariaDB:

```
if [[ ! -d /var/lib/mysql/mysql ]]
then
    ...
fi
```

Каталог `mysql` относится к системной базе данных MariaDB.

Фактическое расположение каталога данных можно определить из самой СУБД запросом:

```
SHOW VARIABLES LIKE 'datadir';
```

Он показывает значение переменной `datadir`, например:

```
/var/lib/mysql/
```

Работать с файлами внутри этого каталога вручную при запущенном сервере MariaDB не рекомендуется. Для переноса, копирования и восстановления баз следует использовать специализированные средства СУБД, например `mysqldump`, `mariadb-dump`, клиент `mysql` и средства резервного копирования MariaDB.

10. Как сделать резервную копию базы данных и затем её восстановить?

Для создания логической резервной копии базы данных можно использовать утилиту `mysqldump`.

Сначала при необходимости создаётся каталог для резервных копий:

```
mkdir -p /var/backup
```

Для создания резервной копии базы `addressbook` выполняется:

```
mysqldump -u root -p addressbook > /var/backup/addressbook.sql
```

mysqldump подключается к MariaDB, получает структуру таблиц и содержащиеся в них данные и формирует последовательность SQL-команд. Оператор:

>

перенаправляет этот вывод в файл /var/backup/addressbook.sql.

Для восстановления данных из полученного файла используется:

```
mysql -u root -p addressbook < /var/backup/addressbook.sql
```

Оператор:

<

передаёт содержимое SQL-файла клиенту mysql, который выполняет содержащиеся в нём команды.

Резервную копию можно сразу сжать программой gzip:

```
mysqldump -u root -p addressbook | gzip > /var/backup/addressbook.sql.gz
```

В данном случае оператор | передаёт вывод mysqldump программе gzip, которая выполняет сжатие.

Для восстановления непосредственно из сжатого файла применяется:

```
zcat /var/backup/addressbook.sql.gz | mysql -u root -p addressbook
```

zcat распаковывает содержимое архива в стандартный поток вывода, а затем через конвейер оно передаётся клиенту MariaDB.

Для хранения нескольких резервных копий удобно включать дату и время создания в имя файла:

```
mysqldump -u root -p addressbook | gzip > $(date +%Y%m%d.%H%M%S.
```

В результате может быть создан файл с именем вида:

```
addressbook.20260929.141500.sql.gz
```

где в имени последовательно указываются год, месяц, день, часы, минуты и секунды.

Следовательно, общий механизм логического резервного копирования и восстановления можно представить следующим образом:

```
MariaDB → mysqldump → SQL-файл → mysql → MariaDB
```

а для сжатого варианта:

```
MariaDB → mysqldump → gzip → .sql.gz  
.sql.gz → zcat → mysql → MariaDB
```

Использование резервных копий позволяет восстановить структуру базы данных и содержащиеся в ней записи после удаления, повреждения данных либо повторного развёртывания сервера.